UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Calidad de Software

Mag. Pablo Virgolini

14 de septiembre de 2026

¿Avanzamos?



¿Cuál es la diferencia entre QA y QC?

QA vs. QC

Bajo este enfoque Calidad no es una sola actividad: se persigue desde dos ángulos distintos y complementarios.

QA — Quality Assurance
Aseguramiento de la calidad

Proactivo. Centrado en el PROCESO

Busca prevenir defectos mejorando la forma en que se construye el software: estándares, revisión de requisitos, capacitación, auditorías. Se pregunta: “¿estamos siguiendo un buen proceso?”

QC — Quality Control
Control de Calidad

Reactivo. Centrado en el PRODUCTO

Busca detectar defectos ya presentes en un artefacto concreto: pruebas de software, inspecciones de código, seguimiento de bugs. Se pregunta: “¿este producto tiene errores?”

QA vs. QC

Dimensión	QA (Aseguramiento)	QC (Control)
Enfoque	**Proceso**	**Producto**
Orientación	Proactiva: prevenir que el defecto ocurra	Reactiva: detectar el defecto ya ocurrido
Pregunta que responde	“¿Estamos trabajando de la forma correcta?”	“¿Este entregable tiene errores?”
Quién lo suele hacer	Roles de QA / gestión de calidad, todo el equipo	Testers, revisores de código, el propio equipo de desarrollo
Ejemplos	Definir estándares, revisar requisitos, auditar el proceso, capacitar	Ejecutar pruebas, inspeccionar código, registrar y seguir bugs

QA mejora cómo trabaja el equipo; QC revisa lo que el equipo produjo. Un buen proyecto necesita las dos.

Trabajamos los conceptos: ¿QA o QC?

Clasifiquemos cada actividad en grupo

1. Ejecutar las pruebas de regresión antes de un release.
2. Definir el estándar de codificación que va a usar el equipo.
3. Revisar el código de un compañero antes de aprobar el merge.
4. Capacitar al equipo en una nueva herramienta de testing.
5. Registrar un bug reportado por un usuario en producción.
6. Auditar si el equipo está siguiendo el proceso ágil definido.
7. Ejecutar un test de carga sobre el checkout de una app.
8. Elaborar el plan de pruebas del proyecto.

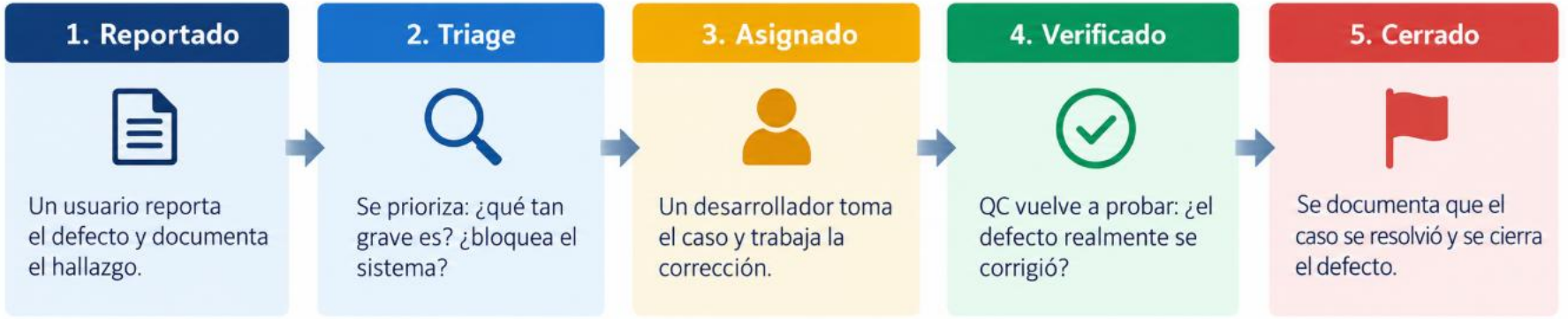
Para cada una: ¿QA (proceso, preventivo) o QC (producto, detección)?

Actividades de Control de Calidad (QC)

El QC trabaja siempre sobre un artefacto concreto ya construido, buscando defectos antes de que lleguen al usuario final.



Ciclo de vida de QC



"Corregido por el desarrollador" no es lo mismo que "verificado". El paso 4 es QC puro: alguien distinto de quien programó la corrección la confirma, con evidencia, que el defecto ya no ocurre.

Actividades de Aseguramiento de Calidad (QA)

El QA trabaja sobre el proceso antes y durante el Desarrollo. No espera a que exista un producto



Definición de estándares

Establecer normas de codificación, documentación, diseño y trabajo que orienten al equipo.



Revisión de procesos

Definir y revisar procedimientos para que la calidad se incorpore durante todo el desarrollo.



Planificación de la calidad

Definir objetivos, responsabilidades, recursos, criterios y estrategia de pruebas.



Auditoría de procesos

Comprobar periódicamente el cumplimiento de los procesos, revisar evidencias y detectar desvíos.



Capacitación del equipo

Desarrollar conocimientos y buenas prácticas para producir software con calidad.



Mejora continua

Analizar métricas, defectos y lecciones aprendidas para ajustar y fortalecer el proceso.

CASO DE ESTUDIO

Boeing 737 MAX

Analizamos que pasó

Resumen del caso

- Boeing re-motorizó el 737 para competir con el Airbus A320neo, sin diseñar un avión nuevo — así conservaba el certificado de tipo y evitaba el reentrenamiento de pilotos.

- El nuevo software MCAS corregía automáticamente el cabeceo, pero dependía de UN solo sensor de ángulo de ataque, sin comparar contra el segundo sensor disponible.

- Dos accidentes (Lion Air, Ethiopian Airlines) con el mismo patrón de falla, con apenas 4 meses de diferencia entre uno y otro.

346
personas fallecidas

20 meses
flota en tierra

+US$20.000M
impacto económico

1
sensor sin redundancia real

Trabajamos en el caso

1 **Lean el caso completo**

De forma individual, antes de la puesta en común grupal.

2 **Formen grupos de 4 participantes**

Cada grupo va a responder las preguntas de los 3 bloques que siguen, con foco en un bloque asignado.

3 **Fundamenten con el texto**

Cada respuesta debe citar o referenciar el párrafo del caso que la sostiene.

4 **Preparen 2 minutos de síntesis**

Un integrante por grupo comparte las conclusiones antes del rescate general.

Entregable: Explicación oral y una hoja con la fundamentación del Bloque asignado

Rescate del caso

Concepto	Cómo se ve en el caso
QC (producto)	MCAS, tomado de forma aislada, funcionaba: hacía exactamente lo que el código decía que tenía que hacer. Ningún test unitario, por sí solo, iba a fallar.
QA (proceso)	El problema nunca estuvo en “¿el código hace lo que dice?”, sino en “¿el proceso consideró los escenarios correctos?”: clasificación de riesgo, revisión de requisitos, independencia de la certificación.

Un producto puede pasar todo el QC y puede exitis un problema grave si el QA (el proceso que decide qué se construye y cómo se valida) tiene fallas de fondo.

Rescate del caso

1 Objetivos de calidad mal definidos

Nunca se fijó como objetivo explícito “cero autoridad de un sistema automático sobre una falla no validada con pilotos reales”.

2 Roles y responsabilidades difusos

La misma organización (Boeing) diseñaba y certificaba: no había una separación clara de quién construye y quién controla.

3 Auditoría externa debilitada

El programa ODA de la FAA delegó la auditoría en la propia Boeing — el equivalente a que una empresa audite su propio balance.

4 Sin revisión ante cambios

Cuando cambió la autoridad de MCAS sobre el avión, nadie volvió a evaluar el riesgo desde cero.

Calidad de Software

Gestión de Pruebas, Riesgos e Incidencias

Niveles de independencias de pruebas

La independencia mejora la objetividad: testers externos a quien escribió el código pueden detectar más defectos, al aportar otra mirada y verificar supuestos que el equipo de desarrollo da por sentados.

1



Sin independencia

El propio desarrollador prueba el código que produjo.

2



Revisión entre pares

Integrantes del equipo prueban el código desarrollado por sus compañeros.

3



Equipo de pruebas independiente

Un equipo especializado, separado del desarrollo, diseña y ejecuta las pruebas.

4



Especialistas externos

Profesionales externos evalúan aspectos específicos, como seguridad, usabilidad o rendimiento.

Más independencia no siempre es mejor. Puede generar aislamiento del equipo de QA, fricción con desarrollo, y cuellos de botella si todo pasa por un único equipo externo antes de avanzar.

Planificación de pruebas

No se realiza una sola vez al inicio: se revisa y ajusta a medida que el proyecto avanza.

1. Definir

- Determinar el alcance, los riesgos y los objetivos.
- Establecer el enfoque general.
- Decidir qué se probará.

2. Organizar

- Integrar y coordinar las actividades.
- Asignar roles y responsabilidades.
- Distribuir los recursos necesarios.

3. Programar y documentar

- Programar el análisis, diseño, implementación, ejecución y evaluación.
- Definir el nivel de detalle y las plantillas de documentación.

4. Medir y ajustar

- Seleccionar métricas de seguimiento y control.
- Supervisar la gestión y resolución de defectos.
- Actualizar el plan cuando sea necesario.

La planificación puede documentarse en un plan maestro y en planes específicos para cada nivel o tipo de prueba.

Estimación del esfuerzo de pruebas

Permite anticipar el tiempo, las personas y los recursos necesarios



Basada en métricas

Parte de evidencia de proyectos comparables.

- Esfuerzo y duración históricos
- Cantidad de casos ejecutados
- Ritmo de ejecución del equipo
- Defectos encontrados en productos similares

Más objetiva cuando existen datos confiables



Basada en expertos

Parte del juicio de quienes conocen el producto y las tareas.

- Complejidad funcional y técnica
- Riesgos y áreas críticas
- Experiencia del equipo
- Incertidumbre y restricciones

Especialmente útil cuando faltan antecedentes



Estimación combinada

La práctica más sólida combina los datos históricos con el juicio del equipo y revisa la estimación cuando aparece nueva información.

Una estimación es una previsión fundamentada, no un compromiso inamovible.

Seguimiento del proceso de prueba

El seguimiento reúne información manual o automática para comparar el avance real con el plan, detectar desvíos y facilitar la retroalimentación.



Preparación

- Casos de prueba diseñados y revisados
- Entorno de pruebas disponible
- Datos de prueba preparados



Ejecución

- Casos ejecutados, aprobados y fallidos
- Casos pendientes o bloqueados
- Avance respecto del cronograma



Defectos

- Cantidad y gravedad
- Estado y tiempo de resolución
- Tendencias y defectos reabiertos



Cobertura y confianza

- Cobertura de requisitos, riesgos o código
- Riesgos pendientes
- Confianza del equipo basada en evidencia



Las métricas no sustituyen el análisis: deben interpretarse junto con el alcance, los riesgos, el tiempo, los costos y los criterios de salida.

Riesgos y pruebas

El riesgo puede definirse como la oportunidad de un evento, peligro, amenaza o situación que sucede y tiene como resultado consecuencias no deseadas, o un problema potencial. El nivel de riesgo está determinado por la probabilidad de que ocurra un evento adverso y su impacto.

Nivel de Riesgo = Probabilidad × Impacto

Por qué importa en pruebas: nunca hay tiempo ni presupuesto para probar todo con la misma intensidad. El análisis de riesgo es lo que permite decidir, con criterio, dónde concentrar el esfuerzo de testing: más pruebas donde el riesgo es alto, menos donde es bajo

Riesgos de proyecto y producto

Riesgos de proyecto



¿Qué puede impedir que el proyecto alcance sus objetivos?
Afectan la planificación, la ejecución o la entrega del trabajo.

- Plazos insuficientes o poco realistas
- Falta de personas, presupuesto o experiencia
- Entorno y datos de prueba no disponibles
- Dependencias externas y demoras de terceros

Riesgos de producto



¿Dónde y cómo puede fallar el software?
Afectan la calidad, el funcionamiento o el impacto del producto.

- Módulos críticos o de alta complejidad
- Vulnerabilidades de seguridad
- Problemas de rendimiento bajo carga
- Fallos de integración entre componentes

Priorización

Riesgo = probabilidad × impacto

Los riesgos más altos reciben mayor atención y profundidad de prueba.

Un buen plan protege ambas dimensiones: un producto confiable necesita pruebas *bien* ejecutadas, y un proyecto puntual necesita un producto con calidad suficiente.

Matriz de riesgo (probabilidad x impacto)

Combina probabilidad e impacto para decidir qué probar primero y con mayor profundidad



Cálculo

Riesgo = probabilidad × impacto

Cada dimensión se valora de 1 a 3.

Leyenda y acción

1–2	Bajo	Pruebas habituales
3–4	Medio	Mayor cobertura y seguimiento
6–9	Alto	Probar primero y con mayor profundidad

La matriz orienta la prioridad, pero la decisión también debe considerar el contexto, los controles existentes y el riesgo aceptable.

Gestión de incidencias



¿Qué es una incidencia?

Es cualquier evento ocurrido durante las pruebas que requiere investigación: una diferencia entre el resultado esperado y el obtenido, o un problema con el entorno, los datos o el propio caso de prueba.



Facilitar la investigación y corrección

Aportar información clara y reproducible para identificar la causa, aislar el problema, corregirlo y verificar la solución.



Dar visibilidad y trazabilidad

Permitir el seguimiento del estado, la prioridad, la responsabilidad y la relación con la versión y el caso de prueba.



Mejorar el proceso

Analizar patrones y causas recurrentes para prevenir problemas y fortalecer continuamente las pruebas y el desarrollo.



Una incidencia no siempre es un defecto del producto

También puede originarse en el entorno, los datos de prueba, una herramienta o un caso de prueba incorrecto.